

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 9



Oleh :

M.YAZEM AGVA ROIZ

NIM 2511533003

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2026

Penulis

DAFTAR ISI

BAB 1.....	4
1. Latar Belakang.....	4
2. Tujuan.....	4
3. Manfaat.....	4
BAB 2.....	5
A. Teori.....	5
B. Code.....	5
a. Node_2511533003.....	5
b. BTree_2511533003.....	10
c. BTreeDriver_251553003.....	12
d. GraphTransversal_2511533003.....	13
C. Tugas.....	15
BAB 3.....	23
a. Kesimpulan.....	23
b. Saran.....	23

BAB 1

PENDAHULUAN

1. Latar Belakang

Dalam pemrograman, struktur suatu program penting dalam mengembangkan suatu aplikasi karena masing masing struktur yang ada memiliki kelebihan dan kekurangan pada setiap kasus yang ditangani. Pemilihan haruslah berdasarkan keputusan yang matang dari resource yang dimiliki dan kasus yang dihadapi, demi mendapatkan performa yang maksimal dan efisien.

Graph dan Tree merupakan salah satu struktur data pada pemrograman yang memungkinkan banyak keterhubungan dengan data dengan data lain. Graph memungkinkan membuat pemetaan dan perutean serta Tree memungkinkan membentuk keterhubungan secara hierarki yang jelas dan mudah untuk dijelajahi.

2. Tujuan

Tujuan dari praktikum adalah memahami dan mengetahui bagaimana alur konsep graph dan tree secara teoritis dan secara prakteknya serta algoritma yang berfungsi dalam graph dan tree.

3. Manfaat

Manfaat dari praktikum adalah mendapatkan pemahaman dan mengetahui bagaimana alur konsep graph dan tree secara teoritis dan secara prakteknya serta algoritma yang berfungsi dalam graph dan tree.

BAB 2

PEMBAHASAN

A. Teori

graph merupakan sebagai sekumpulan titik-titik yang disebut simpul atau node, yang kemudian dihubungkan satu sama lain oleh garis-garis yang disebut sisi atau edge. Setiap simpul mewakili suatu entitas atau objek, seperti seseorang, kota, halaman web, atau variabel. Sementara setiap sisi merepresentasikan adanya hubungan, interaksi, atau keterkaitan antara dua simpul tersebut.

Tree adalah jenis khusus dari graph yang memiliki bentuk menyerupai pohon di dunia nyata, yaitu memiliki satu akar di bagian atas yang kemudian bercabang-cabang ke bawah tanpa pernah membentuk cycle.

B. Code

a. Node_2511533003

```
package pekan9_2511533003;

public class Node_2511533003 {
    int data_3003;
    Node_2511533003 left_3003;
    Node_2511533003 right_3003;
    public Node_2511533003(int data_3003) {
        this.data_3003 = data_3003;
        left_3003 = null;
        right_3003 = null;
    }
}
```

```
public void setLeft_3003(Node_2511533003 node_3003) {  
    if (left_3003 == null) left_3003 = node_3003;  
}
```

```
public void setRight_3003(Node_2511533003 node_3003) {  
    if (right_3003 == null) right_3003 = node_3003;  
}
```

```
public Node_2511533003 getLeft_3003() {  
    return left_3003;  
}
```

```
public Node_2511533003 getRight_3003() {  
    return right_3003;  
}
```

```
public int getData_3003() {  
    return data_3003;  
}
```

```
public void setData_3003(int data_3003) {  
    this.data_3003 = data_3003;  
}
```

```
void printPreorder_3003(Node_2511533003 node_3003) {  
    if (node_3003 == null) return;  
    System.out.print(node_3003.data_3003 + " ");
```

```

        printPreorder_3003(node_3003.left_3003);
        printPreorder_3003(node_3003.right_3003);
    }

void printPostorder_3003(Node_2511533003 node_3003) {
    if (node_3003 == null) return;
    printPostorder_3003(node_3003.left_3003);
    printPostorder_3003(node_3003.right_3003);
    System.out.print(node_3003.data_3003 + " ");
}

void printInorder_3003(Node_2511533003 node) {
    if (node == null) return;
    printInorder_3003(node.left_3003);
    System.out.print(node.data_3003 + " ");
    printInorder_3003(node.right_3003);
}

public String print_3003() {
    return this.print_3003("", true, "");
}

public String print_3003(
    String prefix_3003,
    boolean isTail_3003,
    String sb_3003
) {

```

```

    if (right_3003 != null) {
        right_3003.print_3003(
            prefix_3003 + (isTail_3003 ? "| " : " "),
            false,
            sb_3003
        );
    }

    System.out.println(
        prefix_3003 + (isTail_3003 ? "\\--" : "/--") + data_3003
    );

    if (left_3003 != null) {
        left_3003.print_3003(
            prefix_3003 + (isTail_3003 ? " " : "| "),
            true,
            sb_3003
        );
    }

    return sb_3003;
}
}

```

Code diatas merupakan code ADT yang berfungsi sebagai node yang akan sling disambungkan untuk membentuk tree atau graph. Property inti berupa:

1. Data_3003

Berfungsi Untuk menyimpan data yang berupa int.

2. left_3003

Berfungsi untuk menyimpan keterhubungan dengan node lain pada sisi kiri.

3. right_3003

Berfungsi untuk menyimpan keterhubungan dengan node lain pada sisi kanan.

ADT Node ini juga memiliki method yang berfungsi untuk memungkinkan penjelajahan dengan beberapa metode, diantaranya adalah:

1. printPreorder_3003.

Akan menjelajahi Node satu per satu yang diawali dengan node yang awal lalu langsung maju ke kiri dari node terus seperti hingga tidak ada node kiri untuk dijelajahi maka langsung pindah ke node yang kanan secara berurutan.

2. printPostorder_3003.

Akan menjelajahi graph dengan cara menyusuri semua kedalaman dari kiri dan jika telah sampai di ujung kiri maka baru mengeksekusi data yang ada disana lalu pindah ke kanan pada node tersebut lalu menjelajah hingga paling dalam.

3. printInorder_3003.

Akan menjelajahi graph dengan cara menyusuri semua kedalaman dari kiri dan jika telah sampai di ujung kiri maka baru mengeksekusi data yang ada disana lalu pindah ke kanan pada node tersebut dan bisa langsung mengeksekusi data disana tanpa harus menjelajahi kedalam hingga paling dalam.

b. BTree_2511533003

```
package pekan9_2511533003;
public class BTree_2511533003 {

    private Node_2511533003 root_3003;
    private Node_2511533003 currentNode_3003;

    public BTree_2511533003() {
        this.root_3003 = null;
    }

    private boolean search_3003(Node_2511533003 node_3003, int data_3003) {
        if (node_3003.getData_3003() == data_3003) {
            return true;
        }

        if (node_3003.getLeft_3003() != null) {
            if (search_3003(node_3003.getLeft_3003(), data_3003)) {
                return true;
            }
        }

        if (node_3003.getRight_3003() != null) {
            if (search_3003(node_3003.getRight_3003(), data_3003)) {
                return true;
            }
        }

        return false;
    }

    public boolean search_3003(int data_3003) {
        return search_3003(this.root_3003, data_3003);
    }

    public void printInOrder_3003() {
        this.root_3003.printInorder_3003(this.root_3003);
    }

    public void printPreOrder_3003() {
        this.root_3003.printPreorder_3003(this.root_3003);
    }

    public void printPostOrder_3003() {
        this.root_3003.printPostorder_3003(this.root_3003);
    }

    public Node_2511533003 getRoot_3003() {
        return this.root_3003;
    }
}
```

```

    }

    public boolean isEmpty_3003() {
        return this.root_3003 == null;
    }

    public int countNodes_3003() {
        return this.countNodes_3003(this.root_3003);
    }

    public int countNodes_3003(Node_2511533003 node_3003) {
        int count_3003 = 1;
        if (node_3003 == null) {
            return 0;
        } else {
            count_3003 += countNodes_3003(node_3003.getLeft_3003());
            count_3003 += countNodes_3003(node_3003.getRight_3003());
            return count_3003;
        }
    }

    public void print_3003() {
        this.root_3003.print_3003();
    }

    public Node_2511533003 getCurrent_3003() {
        return currentNode_3003;
    }

    public void setCurrent_3003(Node_2511533003 node_3003) {
        this.currentNode_3003 = node_3003;
    }

    public void setRoot_3003(Node_2511533003 root_3003) {
        this.root_3003 = root_3003;
    }
}

```

Code diatas merupakan kode yang menggunakan Node yang telah dibuat sebelumnya untuk membuat sebuah Tree. Properti utama berupa root_3003 untuk menentukan node yang menjadi ujung akar dari tree lalu currentNode_3003 yang berfungsi untuk menyimpan node saat.

Fungsi utama pada code ini adalah membentuk sebuah tree lalu menggunakan method yang sudah dibuat pada Node untuk melakukan perjalanan setiap node serta print untuk menunjukkan bentuk pohon yang tercipta.

c. BTreeDriver_251553003

```
package pekan9_2511533003;
public class BTreeDriver_251553003 {

    public static void main(String[] args) {
        BTree_2511533003 tree_3003 = new BTree_2511533003();
        System.out.println("Jumlah Simpul Awal Awal Pohon: ");
        System.out.println(tree_3003.countNodes_3003());

        Node_2511533003 root_3003 = new Node_2511533003(1);

        tree_3003.setRoot_3003(root_3003);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_3003.countNodes_3003());

        Node_2511533003 node2_3003 = new Node_2511533003(2);
        Node_2511533003 node3_3003 = new Node_2511533003(3);
        Node_2511533003 node4_3003 = new Node_2511533003(4);
        Node_2511533003 node5_3003 = new Node_2511533003(5);
        Node_2511533003 node6_3003 = new Node_2511533003(6);
        Node_2511533003 node7_3003 = new Node_2511533003(7);
        Node_2511533003 node8_3003 = new Node_2511533003(8);
        Node_2511533003 node9_3003 = new Node_2511533003(9);

        root_3003.setLeft_3003(node2_3003);
        node2_3003.setLeft_3003(node4_3003);
        node2_3003.setRight_3003(node5_3003);
        node4_3003.setRight_3003(node8_3003);
        root_3003.setRight_3003(node3_3003);
        node3_3003.setLeft_3003(node6_3003);
        node3_3003.setRight_3003(node7_3003);
        node6_3003.setLeft_3003(node9_3003);

        tree_3003.setCurrent_3003(tree_3003.getRoot_3003());
        System.out.println("Menampilkan Simpul Terakhir: ");
        System.out.println(tree_3003.getCurrent_3003().getData_3003());
        System.out.println("Jumlah Simpul setelah simpul 7 ditambahkan");
        System.out.println(tree_3003.countNodes_3003());
        System.out.println("inOrder: ");
        tree_3003.printinOrder_3003();
        System.out.println("\npreOrder: ");
        tree_3003.printPreOrder_3003();
        System.out.println("\npostOrder: ");
        tree_3003.printPostOrder_3003();
        System.out.println("Menampilkan Tree Dalam Bnetuk Pohon");
```

```

        tree_3003.print_3003();
    }
}

```

Code diatas merupakan Code driver yang menggunakan Tree yang telah dibuat sebelumnya, dapat dilihat pembentukan Tree menggunakan Node Node secara manual serta penggunaan method yang telah dibuat yang kebanyakan adalah wrapper dari method Node

d. GraphTransversal_2511533003

```

package pekan9_2511533003;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

public class GraphTransversal_2511533003 {

    private Map<String, List<String>> graph_3003 = new HashMap<>();

    public void addEdge_3003(String node1_3003, String node2_3003) {
        graph_3003.putIfAbsent(node1_3003, new ArrayList<>());
        graph_3003.putIfAbsent(node2_3003, new ArrayList<>());
        graph_3003.get(node1_3003).add(node2_3003);
        graph_3003.get(node2_3003).add(node1_3003);
    }

    public void printGraph_3003() {
        System.out.println("Graph awal (Adjacency List):");
        for (String node_3003 : graph_3003.keySet()) {
            System.out.print(node_3003 + "->");
            List<String> neighbors_3003 = graph_3003.get(node_3003);
            System.out.println(String.join(" ", neighbors_3003));
        }

        System.out.println();
    }

    // DFS recursive

```

```

public void dfs_3003(String start_3003) {
    Set<String> visited_3003 = new HashSet<>();
    System.out.println("Penelusuran DFS");
    dfsHelper_3003(start_3003, visited_3003);
    System.out.println();
}

private void dfsHelper_3003(String current_3003, Set<String> visited_3003) {
    if (visited_3003.contains(current_3003)) {
        return;
    }
    visited_3003.add(current_3003);
    System.out.print(current_3003 + " ");
    for (String neighbor_3003 : graph_3003.getDefault(
        current_3003,
        new ArrayList<>()
    )) {
        dfsHelper_3003(neighbor_3003, visited_3003);
    }
}

// BFS
public void bfs_3003(String start_3003) {
    Set<String> visited_3003 = new HashSet<>();
    Queue<String> queue_3003 = new LinkedList<>();
    queue_3003.add(start_3003);
    visited_3003.add(start_3003);
    System.out.println("Penelusuran BFS:");
    while (!queue_3003.isEmpty()) {
        String current_3003 = queue_3003.poll();
        System.out.print(current_3003 + " ");
        for (String neighbor_3003 : graph_3003.getDefault(
            current_3003,
            new ArrayList<>()
        )) {
            if (!visited_3003.contains(neighbor_3003)) {
                queue_3003.add(neighbor_3003);
                visited_3003.add(neighbor_3003);
            }
        }
    }
    System.out.println();
}

public static void main(String[] args) {
    GraphTransversal_2511533003 graph_3003 =
        new GraphTransversal_2511533003();
    graph_3003.addEdge_3003("A", "B");
    graph_3003.addEdge_3003("A", "C");
}

```

```

graph_3003.addEdge_3003("B", "D");
graph_3003.addEdge_3003("B", "E");
System.out.println("Graph Awal Adalah: ");
graph_3003.printGraph_3003();
graph_3003.dfs_3003("A");
graph_3003.bfs_3003("A");
    }
}

```

Code diatas merupakan kode yang menggunakan Node dengan tujuan membentuk graph, terdapat method penting antara lain:

1. dfs_3003

Berfungsi untuk melakukan penjelajahan secara DFS menggunakan konsep stacking

2. bfs_3003

Berfungsi untuk melakukan penjelajahan secara DFS menggunakan konsep Queue

C. Tugas

DESKRIPSI

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph. Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa:

1. Gedung Perkuliahan
2. Fakultas
3. Universitas
4. Sekolah
5. Rumah Sakit
6. Mall

7. Tempat Wisata
8. Bandara
9. Atau lokasi lainnya

PENGERJAAN

A. MyNode_2511533003.java
package pekan9_2511533003;

```
import java.util.ArrayList;
```

```
class MyNode_2511533003 {
```

```
    String label_3003;
```

```
    ArrayList<MyNode_2511533003> childs_3003;
```

```
    public MyNode_2511533003(String name_3003) {
```

```
        this.label_3003 = name_3003;
```

```
        this.childs_3003 = new ArrayList<>();
```

```
    }
```

```
}
```

Code ini merupakan Code ADT yang akan digunakan sebagai node untuk membentuk graph. Memiliki 2 properti penting diantaranya adalah:

1. label, yang berfungsi sebagai penamaan dari node
2. childs, yang berfungsi menyimpan relasi dengan node lainnya

B. PetaKampus_2511533003.java

Merupakan program GUI inti pada pengerjaan tugas ini, diawali dengan method yang berfungsi menginisialisasi map yaitu initial_3003.

```
void initial_3003() {
```

```
    map_3003 = new HashMap<>();
```

```
    MyNode_2511533003 gedungPerkuliahan_3003 = new MyNode_2511533003(
```

```
        "Gedung Perkuliahan"
```

```
    );
```

```
    MyNode_2511533003 fakultas_3003 = new MyNode_2511533003("Fakultas");
```



```
MyNode_2511533003 universitas_3003 = new MyNode_2511533003(
    "Universitas"
);
MyNode_2511533003 sekolah_3003 = new MyNode_2511533003("Sekolah");
MyNode_2511533003 rumahSakit_3003 = new MyNode_2511533003(
    "Rumah Sakit"
);
MyNode_2511533003 mall_3003 = new MyNode_2511533003("Mall");
MyNode_2511533003 tempatWisata_3003 = new MyNode_2511533003(
    "Tempat Wisata"
);
MyNode_2511533003 bandara_3003 = new MyNode_2511533003("Bandara");

gedungPerkuliahan_3003.chlds_3003.add(universitas_3003);
gedungPerkuliahan_3003.chlds_3003.add(tempatWisata_3003);
gedungPerkuliahan_3003.chlds_3003.add(fakultas_3003);

fakultas_3003.chlds_3003.add(gedungPerkuliahan_3003);
fakultas_3003.chlds_3003.add(tempatWisata_3003);

tempatWisata_3003.chlds_3003.add(gedungPerkuliahan_3003);
tempatWisata_3003.chlds_3003.add(fakultas_3003);
tempatWisata_3003.chlds_3003.add(sekolah_3003);
tempatWisata_3003.chlds_3003.add(rumahSakit_3003);

rumahSakit_3003.chlds_3003.add(tempatWisata_3003);
rumahSakit_3003.chlds_3003.add(mall_3003);
```

```

sekolah_3003.childs_3003.add(universitas_3003);
sekolah_3003.childs_3003.add(tempatWisata_3003);
sekolah_3003.childs_3003.add(mall_3003);

mall_3003.childs_3003.add(sekolah_3003);
mall_3003.childs_3003.add(rumahSakit_3003);
mall_3003.childs_3003.add(bandara_3003);

bandara_3003.childs_3003.add(universitas_3003);
bandara_3003.childs_3003.add(mall_3003);

universitas_3003.childs_3003.add(gedungPerkuliahan_3003);
universitas_3003.childs_3003.add(bandara_3003);

map_3003.put("Gedung Perkuliahan", gedungPerkuliahan_3003);
map_3003.put("Fakultas", fakultas_3003);
map_3003.put("Universitas", universitas_3003);
map_3003.put("Sekolah", sekolah_3003);
map_3003.put("Rumah Sakit", rumahSakit_3003);
map_3003.put("Mall", mall_3003);
map_3003.put("Tempat Wisata", tempatWisata_3003);
map_3003.put("Bandara", bandara_3003);
}

```

Lalu untuk melakukan pencarian BFS, menggunakan method bfs_3003, yang akan menjelajahi secara menyeluruh.

```

void bfs_3003(MyNode_2511533003 node_3003, String tujuan_3003) {
    tahapan_3003 = 0;

```

```

Map<String, String> visited_3003 = new HashMap<>();
Queue<MyNode_2511533003> queue_3003 = new LinkedList<>();
queue_3003.add(node_3003);

while (queue_3003.size() > 0) {
    tahapan_3003 += 1;
    MyNode_2511533003 temp_3003 = queue_3003.poll();
    if (visited_3003.get(temp_3003.label_3003) != null) {
        continue;
    }

    visited_3003.put(temp_3003.label_3003, temp_3003.label_3003);
    route_3003 += " " + temp_3003.label_3003 + " |";
    if (temp_3003.label_3003 == tujuan_3003) {
        break;
    }

    int childs_size_3003 = temp_3003.childs_3003.size();

    for (int i_3003 = 0; i_3003 < childs_size_3003; i_3003++) {
        queue_3003.add(temp_3003.childs_3003.get(i_3003));
    }
}
}

```

Lalu untuk melakukan pencarian DFS, menggunakan method dfs_3003, yang akan menjelajahi secara mendalam terlebih dahulu.

```

void dfs_3003(MyNode_2511533003 node_3003, String tujuan_3003) {

```

```

tahap_3003 = 0;

Map<String, String> visited_3003 = new HashMap<>();

Stack<MyNode_2511533003> stack_3003 = new Stack<>();

stack_3003.push(node_3003);

while (stack_3003.size() > 0) {
    tahap_3003 += 1;

    MyNode_2511533003 temp_3003 = stack_3003.pop();

    if (visited_3003.get(temp_3003.label_3003) != null) {
        continue;
    }

    visited_3003.put(temp_3003.label_3003, temp_3003.label_3003);

    route_3003 += " " + temp_3003.label_3003 + " |";

    if (temp_3003.label_3003 == tujuan_3003) {
        break;
    }

    int childs_size_3003 = temp_3003.childs_3003.size();

    for (int i_3003 = 0; i_3003 < childs_size_3003; i_3003++) {
        stack_3003.push(temp_3003.childs_3003.get(i_3003));
    }
}
}

```

Lalu method reset_3003 untuk melakukan reset.

```

void reset_3003() {
    initial_3003();
}

```

```

tahap_3003 = 0;

done_3003 = false;

hasilPencarian_3003.setText("");

nodeDikunjungi_3003.setText("");

textField_3003.setText("");

route_3003 = "";

}

```

Saat pertama kali dijalankan, akan menampilkan sebuah GUI yang memiliki tampilan seperti pada gambar dibawah ini.

The GUI is a Java Swing window with a dark title bar. It contains two dropdown menus for selecting start and end locations, three buttons for search algorithms, a network diagram, and three text input fields for search results.

Terdapat opsi untuk memilih lokasi awal dan lokasi tujuan, untuk contoh maka pilih lokasi awal gedung kuliah dan lokasi tujuan adalah Mall. lalu gunakan opsi DFS atau BFS untuk melakukan pencarian.

lokasi awal **Gedung Perkuliah...**

lokasi tujuan **Mall**

DFS **BFS** **RESET**

Hasil Pencarian
Mall

Node Dikunjungi
Gedung Perkuliahan | Fakultas | Tempat Wisata | Rumah Sakit | Mall |

Node Dikunjungi
5

lokasi awal **Gedung Perkuliah...**

lokasi tujuan **Mall**

DFS **BFS** **RESET**

Hasil Pencarian
Mall

Node Dikunjungi
Perkuliahan | Universitas | Tempat Wisata | Fakultas | Bandara | Sekolah | Rumah Sakit | Mall |

Node Dikunjungi
14

Lalu gunakan Reset untuk mengembalikan ke awal lagi

BAB 3

PENUTUPAN

a. Kesimpulan

Graph dan Tree adalah struktur data yang memiliki keunggulan dalam pemetaan dan pembentukan hirarki data. Tree dan Graph digunakan dalam perutean dan penyimpanan data yang memiliki kemampuan yaitu keterhubungan yang memungkinkan pencarian yang lebih masif dan eksploratif menggunakan algoritma algoritma yang ada demi mendapatkan performa yang maksimal.

b. Saran

Laporan praktikum yang telah penulis tuliskan, masih memiliki kekurangan. Penulis sendiri pun juga membuka saran dan kritikan untuk meningkatkan kualitas laporan pratikum ini.